

Relatório de Laboratório

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab03
Grupo (trio)	Amanda Bicalho · Guilherme de Almeida · Pedro Duarte
Link do repositório / GitHub Projects	https://github.com/guilhermeas04/Laboratorio-de-experimentacao-de-software
Data de entrega	27/08/2026

1. Introdução

As métricas DORA são amplamente utilizadas para avaliar o desempenho da entrega de software, principalmente a partir de duas dimensões complementares: velocidade e estabilidade. Essas métricas ajudam equipes e organizações a entender com que frequência entregam novas versões, quanto tempo uma mudança leva para chegar aos usuários, com que frequência as entregas apresentam problemas e quanto tempo é necessário para recuperar o sistema após uma falha. Por esse motivo, tornaram-se uma referência importante para a avaliação de práticas de desenvolvimento e entrega contínua na Engenharia de Software. Entretanto, aplicar essas métricas a repositórios open source apresenta algumas limitações. O GitHub, por exemplo, não registra necessariamente todos os deploys realizados em produção nem permite identificar de forma direta quais falhas chegaram aos usuários finais. Por isso, neste laboratório utilizamos releases publicadas como uma aproximação para deploys e execuções do GitHub Actions como representação da atividade de integração e entrega contínua. O objetivo é analisar dados públicos de repositórios reais ao longo de uma janela de doze meses, investigando não apenas os valores das métricas DORA, mas também as limitações e a confiabilidade das aproximações utilizadas.

RQ1. Qual é a frequência de deploys dos repositórios populares que usam CI/CD? **Hipótese informal:** esperamos que a frequência de deploy varie significativamente entre os repositórios e seja baixa ou moderada na maior parte da amostra, devido às diferenças entre bibliotecas, ferramentas e aplicações.

RQ2. Qual é o tempo entre um commit e seu respectivo deploy? **Hipótese informal:** esperamos que o lead time apresente uma distribuição assimétrica, com grande parte dos commits chegando a uma release em poucos dias, enquanto alguns commits mais antigos aumentam o tempo observado em determinadas releases.

RQ3. Qual é a taxa de falha das mudanças entregues por esses repositórios? **Hipótese informal:** esperamos encontrar diferenças entre a taxa de falha calculada a partir de execuções de CI e aquela inferida por releases corretivas, uma vez que uma falha em um pipeline não representa necessariamente uma falha em produção.

RQ4. Qual é o tempo de recuperação após uma execução de CI/CD com falha? **Hipótese informal:** esperamos que a maioria dos episódios de falha seja resolvida em poucas horas, embora alguns casos apresentem períodos de recuperação mais longos ou não possam ser completamente observados dentro da janela analisada.

RQ5. Repositórios com maior frequência de deploy apresentam maior ou menor taxa de falha? **Hipótese informal:** esperamos encontrar uma associação negativa ou próxima de zero entre frequência de deploy e taxa de falha, considerando que uma maior velocidade de entrega não implica necessariamente menor estabilidade.

RQ6. Quais características dos repositórios estão associadas a um melhor desempenho DORA? **Hipótese informal:** esperamos que características como linguagem principal, popularidade, idade do repositório e tamanho da comunidade apresentem associação com algumas métricas DORA, mas que nenhum fator isolado seja suficiente para explicar todas as diferenças de desempenho observadas.

RQ7. O quanto a classificação DORA de um repositório depende da definição operacional escolhida? **Hipótese informal:** esperamos que uma parcela relevante dos repositórios mude de classificação quando forem consideradas diferentes formas de cálculo, como inclusão de pré-releases, uso de tags

ou variações na definição de lead time e taxa de falha.

2. Contexto

Este estudo corresponde ao Laboratório 03 da disciplina de Laboratório de Experimentação de Software, etapa em que o grupo transforma conceitos de medição de software em uma coleta automatizada e reproduzível. Ele sucede os laboratórios anteriores no percurso acadêmico, mas possui escopo e dados próprios: não reutiliza os dados do LAB2 nem depende do Kanban ou de artefatos de outro experimento. A conexão com os trabalhos anteriores está no processo experimental, que mantém a prática de formular questões, definir variáveis, automatizar a coleta, registrar decisões e preservar evidências para análise e replicação. O objeto de estudo são repositórios públicos e populares do GitHub que utilizam GitHub Actions. Para cada repositório, observamos releases publicadas, tags, commits entre releases e execuções de workflows no branch padrão durante uma janela de doze meses. Esses dados permitem estimar as quatro dimensões centrais de desempenho de entrega: frequência de deploy, lead time for changes, change failure rate e tempo de recuperação. Como o GitHub não informa diretamente todos os deploys em produção, releases e execuções de CI são tratados como proxies e suas limitações são registradas na análise. A base conceitual das métricas é o modelo DORA apresentado por Forsgren, Humble e Kim em *Accelerate: The Science of Lean Software and DevOps* (2018), complementado pelas definições atuais do projeto DORA. O planejamento das perguntas, métricas e procedimentos também segue a lógica Goal–Question–Metric (GQM), proposta por Basili, Caldiera e Rombach, que relaciona objetivos de medição a questões observáveis e métricas operacionais. Essa combinação orienta a passagem do objetivo geral de avaliar desempenho de entrega para as RQs e os indicadores calculados neste laboratório.

3. Metodologia

3.1 Principais Desafios

Os principais desafios do laboratório estiveram relacionados ao volume e à confiabilidade da coleta realizada pela API GraphQL do GitHub. A consulta reúne várias informações agregadas para cada repositório, como Pull Requests aceitas, releases, issues totais e fechadas, linguagem e datas de atividade. Ao solicitar 50 ou 100 repositórios por requisição, a API apresentou erros 502 Bad Gateway, exigindo a adoção de páginas menores, com 10 repositórios, além de retentativas e intervalos entre chamadas. Essa decisão tornou a coleta dos 1.000 repositórios mais demorada, mas aumentou sua estabilidade. Outro desafio foi garantir que a paginação resultasse em exatamente 1.000 repositórios únicos, o que exigiu controle de cursores e detecção de duplicidades. Também foi necessário lidar com campos ausentes, como linguagem primária e datas de último push, e com pequenas diferenças de horário capazes de produzir valores negativos no cálculo de dias, corrigidas por normalização. Do ponto de vista metodológico, identificou-se que `updatedAt` não representa necessariamente uma alteração no código, pois pode mudar em decorrência de atividades em issues e Pull Requests; por isso, `pushedAt` foi incluído como variável complementar. A presença de distribuições muito assimétricas e valores extremos também dificultou a interpretação das médias, levando ao uso de medianas, quartis e detecção de outliers por IQR. Por fim, o GitHub Projects não disponibiliza um histórico completo das mudanças de Status por meio da API, tornando necessária a geração periódica de snapshots em CSV para preservar o estado do Kanban ao final de cada sprint.

3.2 Tomadas de Decisão

O grupo definiu como amostra os 1.000 repositórios retornados pela busca do GitHub ordenada de forma decrescente pelo número de estrelas, sem excluir projetos por linguagem, finalidade ou ausência de determinadas métricas, pois essas ausências também representam características relevantes da população estudada. Para garantir exatamente 1.000 repositórios únicos, a coleta utiliza paginação por cursor e elimina duplicidades por `nameWithOwner`. Embora páginas maiores reduzissem o tempo total, consultas com 50 ou 100 itens apresentaram erros de gateway; por isso, foram adotadas páginas de 10 repositórios, priorizando estabilidade em troca de maior duração. A implementação foi realizada em Python com requisições HTTP diretas à API GraphQL, sem bibliotecas específicas para acesso ao GitHub, respeitando a restrição do laboratório e facilitando o processamento e a exportação em CSV. As métricas foram mantidas de forma uniforme em toda a amostra: idade calculada em dias desde `createdAt`, contribuição externa representada pelo total de Pull Requests no estado MERGED, releases medidas por `totalCount`, linguagem obtida de `primaryLanguage` e percentual de issues fechadas calculado pela divisão entre issues fechadas e totais. Para atualização, decidiu-se conservar `updatedAt`, conforme o enunciado, mas acrescentar `pushedAt`, por representar melhor alterações no conteúdo. Valores ausentes foram documentados e mantidos quando válidos, como repositórios sem linguagem ou issues, enquanto valores impossíveis, duplicidades e inconsistências interrompem a execução. Devido à assimetria das distribuições, a mediana foi priorizada e os outliers foram identificados pelo método IQR, mas não removidos. No processo de trabalho, estabeleceu-se limite WIP de três Issues na coluna Doing, equivalente a uma tarefa simultânea por integrante. Esse limite permite trabalho paralelo no trio sem incentivar o

acúmulo individual; ao ser atingido, uma tarefa deve avançar para Review ou Done antes que outra seja iniciada.

3.3 Etapas

O desenvolvimento do Laboratório 01 foi organizado em três sprints e acompanhado por meio de Issues no GitHub Projects. Cada atividade foi atribuída a um integrante, movimentada no quadro conforme seu andamento e associada aos respectivos commits e Pull Requests. Na Lab01S01, foi preparada a estrutura inicial do repositório, implementada a consulta GraphQL para os 100 repositórios mais populares, desenvolvidos os primeiros validadores das questões de pesquisa e realizada a integração da coleta. Na Lab01S02, a consulta foi ampliada para 1.000 repositórios por meio de paginação, os dados foram exportados para CSV, as métricas foram validadas sobre a base completa, as análises das RQ01 a RQ07 foram integradas e foi gerado o primeiro snapshot do GitHub Project. Na Lab01S03, foi desenvolvido um pipeline comum de visualização, foram produzidos os gráficos das sete questões de pesquisa, os scripts foram integrados em um único ponto de entrada e foi implementada a validação automática das figuras e dos resultados. Por fim, os resultados, limitações, contribuições adicionais e evidências do processo foram consolidados neste relatório.

Sprint	Entrega	Responsável	Issue
Lab01S01	Configuração do repositório e do GitHub Projects	Guilherme de Almeida Santos	#10
Lab01S01	Implementação e validação das métricas de RQ01 e RQ02	Guilherme de Almeida Santos	#11
Lab01S01	Implementação e validação das métricas de RQ03 e RQ04	Pedro Rodrigues Duarte	#12
Lab01S01	Implementação e validação das métricas de RQ05 e RQ06	Amanda Bicalho Silva	#13
Lab01S01	Integração das extrações em uma consulta GraphQL única	Pedro Rodrigues Duarte	#14
Lab01S01	Automatização da consulta dos 100 repositórios	Amanda Bicalho Silva	#15
Lab01S02	Paginação GraphQL e exportação de 1.000 repositórios	Pedro Rodrigues Duarte	#20

	em CSV		
Lab01S02	Validação das RQ01 e RQ02 nos 1.000 repositórios	Guilherme de Almeida Santos	#21
Lab01S02	Validação das RQ03 e RQ04 nos 1.000 repositórios	Pedro Rodrigues Duarte	#22
Lab01S02	Validação das RQ05, RQ06 e RQ07 nos 1.000 repositórios	Amanda Bicalho Silva	#23
Lab01S02	Integração das validações e análises das RQ01 a RQ07	Guilherme de Almeida Santos	#24
Lab01S02	Implementação e exportação do snapshot do GitHub Project	Amanda Bicalho Silva	#25
Lab01S02	Revisão da entrega e atualização do GitHub Project	Guilherme de Almeida Santos	#26
Lab01S03	Preparação do pipeline comum de análise e visualização	Pedro Rodrigues Duarte	#33
Lab01S03	Análise e visualização das RQ01 e RQ02	Guilherme de Almeida Santos	#34
Lab01S03	Análise e visualização das RQ03 e RQ04	Pedro Rodrigues Duarte	#35
Lab01S03	Análise e visualização das RQ05, RQ06 e RQ07	Amanda Bicalho Silva	#36
Lab01S03	Integração da geração das visualizações de RQ01 a RQ07	Guilherme de Almeida Santos	#37
Lab01S03	Validação automática das figuras e dos resultados	Amanda Bicalho Silva	#38

O grupo utiliza o GitHub Projects, vinculado ao repositório do laboratório, para acompanhar o desenvolvimento durante o semestre. Os cartões do quadro correspondem a Issues reais do repositório e possuem um responsável definido no campo Assignee. O fluxo adotado é composto pelas colunas Backlog → To Do → Doing → Review → Done. As tarefas inicialmente planejadas permanecem em Backlog; as selecionadas para a sprint avançam para To Do; atividades iniciadas são movidas para Doing; entregas aguardando verificação ficam em Review; e somente tarefas implementadas e revisadas são movidas para Done.

A coluna Doing possui limite de WIP de três Issues simultâneas, equivalente a uma atividade em andamento por integrante do trio. Essa política possibilita o desenvolvimento paralelo sem permitir que uma mesma pessoa acumule várias tarefas abertas. Quando as três vagas estão ocupadas, uma Issue precisa avançar para Review ou Done antes que outra tarefa seja iniciada. O grupo também utiliza snapshots em CSV para preservar o estado do Project ao final de cada sprint, pois o GitHub Projects não disponibiliza pela API um histórico completo das movimentações entre colunas.

3.4 Ferramentas

A coleta, o processamento e a validação dos dados foram realizados por scripts próprios desenvolvidos pelo grupo. Nenhuma biblioteca específica para consulta à API do GitHub foi utilizada. As ferramentas empregadas estão descritas a seguir.

Ferramenta	Versão ou tecnologia	Utilização
Python	3.13.7	Implementação dos scripts de coleta, normalização, validação, análise e exportação
GitHub GraphQL API	GraphQL v4	Consulta paginada dos 1.000 repositórios e exportação dos itens do GitHub Projects
Requests	Biblioteca Python	Envio direto das requisições HTTP para o endpoint GraphQL do GitHub
python-dotenv	Biblioteca Python	Leitura segura das variáveis de ambiente armazenadas no arquivo .env
csv	Biblioteca padrão do Python	Exportação e leitura dos dados consolidados e snapshots
statistics	Biblioteca padrão do Python	Cálculo de média, mediana, quartis e limites utilizados na identificação de outliers
Datetime	Biblioteca padrão do Python	Cálculo da idade dos repositórios e dos dias desde a última atualização ou push
Git	Sistema de controle de versão	Versionamento dos scripts, relatórios, dados e snapshots
Github	Plataforma de hospedagem	Armazenamento do repositório, revisão por Pull Requests e rastreabilidade entre

		Issues e commits
Github Projects v2	Ferramenta de gestão	Organização das atividades em Kanban e acompanhamento de Assignees e Status
Markdown	Formato textual	Produção da documentação, relatórios e registros das validações
CSV	Formato de dados	Armazenamento dos 1.000 repositórios coletados e dos snapshots do Project
GitHub Octoverse 2024	Fonte de referência	Definição do conjunto de linguagens consideradas mais populares na RQ05
Matplotlib	Biblioteca Python	Geração dos histogramas, boxplots e gráficos de dispersão das RQ01 a RQ07
Seaborn	Biblioteca Python	Aplicação de estilo comum e construção de visualizações estatísticas
Backend Agg	Backend do Matplotlib	Geração automática de figuras PNG sem interface gráfica
PNG	Formato de imagem	Armazenamento reproduzível das visualizações geradas pelo pipeline

A consulta dos repositórios foi realizada diretamente no endpoint <https://api.github.com/graphql>. O cliente HTTP foi implementado pelo próprio grupo com autenticação por token, tratamento de erros, tentativas e tempo máximo de resposta. A paginação utiliza os campos pageInfo, endCursor e hasNextPage, coletando os repositórios em páginas de 10 itens para reduzir erros de gateway provocados pelo custo da consulta agregada.

A coleta, a normalização e a validação dos dados foram implementadas em Python, utilizando bibliotecas da distribuição padrão, além de requests e python-dotenv. As visualizações foram produzidas com Matplotlib e Seaborn, configuradas com o backend não interativo Agg, permitindo a geração automática das figuras sem abertura de interface gráfica ou edição manual. Não foram utilizadas bibliotecas específicas para acesso à API do GitHub; as requisições GraphQL foram enviadas diretamente ao endpoint oficial. Os dados consolidados foram armazenados em CSV e processados pelos scripts desenvolvidos pelo próprio grupo.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Diferença entre a data e hora da coleta e createdAt: idade = data_coleta – createdAt	Dias	Script Python próprio e GitHub GraphQL API
RQ02	Pull Requests aceitas	Valor de pullRequests(states: MERGED).totalCount, considerando somente Pull	Quantidade de PRs	Script Python próprio e GitHub GraphQL API

		Requests no estado MERGED		
RQ03	Total de releases	Valor de releases.totalCount, correspondente ao total acumulado de releases registradas no GitHub	Quantidade de releases	Script Python próprio e GitHub GraphQL API
RQ04	Tempo desde a última atualização	Diferença entre a data da coleta e updatedAt: tempo_atualizacao = data_coleta – updatedAt	Dias	Script Python próprio e GitHub GraphQL API
RQ04 - adicional	Tempo desde o último push	Diferença entre a data da coleta e pushedAt: tempo_push = data_coleta – pushedAt	Dias	Script Python próprio e GitHub GraphQL API
RQ05	Linguagem primária	Valor de primaryLanguage.name; quando o GitHub não identifica uma linguagem, o registro é classificado como “sem linguagem identificada”	Categoria nominal	GitHub GraphQL API e GitHub Octoverse 2024
RQ05	Presença no conjunto de linguagens populares	Comparação de primaryLanguage.name com o top 10 do GitHub Octoverse 2024: Python, JavaScript, TypeScript, Java, C#, C++, PHP, Shell, C e Go	Sim/Não e contagem por linguagem	Script Python próprio e GitHub Octoverse 2024
RQ06	Proporção de issues fechadas	closedIssues.totalCount ÷ issues.totalCount; quando o total de issues é zero, a razão é registrada como não calculável	Proporção entre 0 e 1 e percentual	Script Python próprio e GitHub GraphQL API
RQ07	Contribuição externa por linguagem	Agrupamento dos repositórios por primaryLanguage.name e cálculo da mediana e média de pullRequests(states: MERGED).totalCount em cada grupo	PRs aceitas por linguagem	Script Python próprio e GitHub GraphQL API
RQ07	Releases por linguagem	Agrupamento dos repositórios por primaryLanguage.name e cálculo da mediana e média de releases.totalCount em cada grupo	Releases por linguagem	Script Python próprio e GitHub GraphQL API
RQ07	Atualização por linguagem	Agrupamento dos repositórios por primaryLanguage.name e cálculo da mediana e média dos dias desde updatedAt e, complementarmente, desde pushedAt	Releases por linguagem	Script Python próprio e GitHub GraphQL API

Qualidade dos dados — adicional	Possíveis outliers	Para cada métrica numérica, cálculo de $IQR = Q3 - Q1$; são classificados como possíveis outliers os valores menores que $Q1 - 1,5 \times IQR$ ou maiores que $Q3 + 1,5 \times IQR$	Quantidade de outliers	Script Python próprio e módulo statistics
Qualidade dos dados — adicional	Integridade da base	Verificação de exatamente 1.000 nomes únicos, presença dos campos obrigatórios, datas processáveis, contagens não negativas e consistência de $closed_issues \div total_issues$	Contagens de erros e ausentes	Validadores próprios em Python

3.6 Inovações Propostas pelo Grupo (30% da nota)

Como contribuição adicional aos requisitos do enunciado, o grupo incorporou variáveis e procedimentos metodológicos destinados a melhorar a confiabilidade da coleta e a interpretação dos resultados. Essas inovações não substituem as métricas obrigatórias das RQ01 a RQ07, mas complementam a análise e reduzem ameaças à validade do estudo.

A primeira inovação foi a inclusão do campo `pushedAt` como variável complementar para as RQ04 e RQ07. O enunciado define a frequência de atualização a partir de `updatedAt`, mas esse campo pode ser modificado por atividades que não representam alterações no conteúdo do repositório, como movimentações em Issues, Pull Requests, wiki ou configurações. Para obter uma medida mais próxima da atualização do código, o grupo coletou também `pushedAt` e calculou a quantidade de dias desde o último push. Os resultados dessa variável serão apresentados na seção de Resultados, em conjunto com a RQ04 e com os agrupamentos por linguagem da RQ07. Na Discussão e na Conclusão, serão comparados os resultados produzidos por `updatedAt` e `pushedAt`, destacando as diferenças entre atividade geral e atualização do conteúdo.

A segunda inovação foi a criação de um pipeline automático de validação da qualidade dos dados. Antes das análises, o programa verifica se foram coletados exatamente 1.000 repositórios únicos, identifica duplicidades, contabiliza campos ausentes, valida datas, rejeita contagens negativas e confere se a proporção de issues fechadas corresponde à divisão entre issues fechadas e totais. Essa abordagem foi considerada relevante porque erros de paginação, valores ausentes e inconsistências poderiam alterar médias, medianas e agrupamentos. Os resultados dessas verificações aparecem na seção de Coleta de Dados e nos relatórios de validação das RQs. Na Discussão, essas evidências serão utilizadas para avaliar a confiabilidade da base analisada.

A terceira inovação foi a aplicação do método do intervalo interquartil para identificar possíveis outliers. Para cada métrica numérica, foi calculado $IQR = Q3 - Q1$, classificando como possíveis outliers valores inferiores a $Q1 - 1,5 \times IQR$ ou superiores a $Q3 + 1,5 \times IQR$. Esses registros não foram removidos, pois representam projetos reais com características excepcionais, mas foram documentados e analisados separadamente. Essa decisão permite explicar diferenças elevadas entre média e mediana, especialmente nas métricas de Pull Requests e releases. Os outliers serão

apresentados nos Resultados, discutidos na comparação entre hipóteses e evidências e considerados nas limitações descritas na Conclusão.

Por fim, nas análises da RQ07, o grupo identificou grupos de linguagens com menos de cinco repositórios e os marcou como grupos de baixa representatividade. Essa segmentação evita que resultados obtidos com poucos projetos sejam interpretados com o mesmo grau de confiança dos grupos mais numerosos. A quantidade de repositórios por linguagem será apresentada juntamente com as métricas agrupadas, e a influência dos grupos pequenos será considerada na Discussão e na Conclusão.

4. Resultados

4.1 Coleta de Dados

A coleta final foi executada em 18 de agosto de 2026 e produziu uma base com exatamente 1.000 repositórios populares do GitHub. Após as verificações de qualidade, todos os 1.000 registros foram mantidos, pois não foram identificados repositórios duplicados, nomes ausentes, datas inválidas ou contagens numéricas negativas. A base contém 1.000 valores válidos para idade, Pull Requests aceitas, releases e datas de atualização. Os repositórios analisados foram criados entre 11 de abril de 2008 e 13 de agosto de 2026, representando projetos com diferentes níveis de maturidade. A coleta foi realizada por paginação GraphQL em grupos de 10 repositórios, até atingir o total previsto.

Verificação	Resultado
Repositórios coletados	1.000
Repositórios únicos	1.000
Repositórios mantidos após validação	1.000
Repositórios duplicados	0
Nomes ausentes	0
Datas de criação ausentes ou inválidas	0
Idades ausentes ou inválidas	0
Contagens de Pull Requests ausentes ou inválidas	0
Valores de releases ausentes ou inválidos	0
Datas de último push ausentes	0
Linguagens primárias ausentes	87
Repositórios sem issues	43
Proporções de issues inválidas ou inconsistentes	0

A ausência de linguagem primária em 87 repositórios foi mantida e representada pela categoria “sem linguagem identificada”, pois muitos desses projetos correspondem a listas, documentações, materiais

educacionais ou outros conteúdos que não possuem uma linguagem predominante detectada pelo GitHub. Para a RQ06, 43 repositórios não possuíam issues; nesses casos, a proporção de issues fechadas não foi calculada, evitando divisão por zero. Dessa forma, a análise dessa métrica considerou 957 registros válidos. Esses casos não foram removidos da base geral, pois representam características reais dos repositórios estudados.

As distribuições apresentaram valores extremos, especialmente nas métricas de contribuição e releases. Pelo método IQR, foram identificados 124 possíveis outliers de Pull Requests aceitas, 92 de releases, 8 de dias desde updatedAt, 194 de dias desde pushedAt e 38 da proporção de issues fechadas. Não foram identificados outliers de idade. Esses valores foram mantidos porque correspondem a projetos reais e relevantes, e não a erros de coleta. Para reduzir a influência dos extremos, a mediana e os quartis foram priorizados na interpretação, enquanto a média foi apresentada como medida complementar.

Na análise por linguagem, grupos com menos de cinco repositórios foram marcados como grupos de baixa representatividade e interpretados com cautela. Esse tratamento foi aplicado a linguagens como Clojure, MDX, PHP, Vim Script, Zig, Astro, Dockerfile, Haskell, PowerShell e outras categorias com poucos projetos. Entre os 1.000 repositórios, 724 utilizam uma linguagem pertencente ao top 10 adotado a partir do GitHub Octoverse 2024.

Para o acompanhamento do processo de desenvolvimento, foi gerado em 19 de agosto de 2026 o primeiro snapshot do GitHub Projects. O arquivo contém 13 Issues correspondentes às Sprints 1 e 2, todas com responsável definido e Status Done. Como este laboratório não envolve trials experimentais, essa medida não se aplica à coleta. O snapshot foi preservado em CSV para registrar o estado do Kanban no encerramento da Lab01S02 e constituir a base histórica que será utilizada nos laboratórios posteriores.

4.2 Visualização Gráfica

4.2.1 RQ01 — Idade dos repositórios

RQ01: Sistemas populares são maduros/antigos?

A idade de cada repositório foi calculada pela diferença, em dias, entre a data da coleta e o campo createdAt. A Figura 1 apresenta a distribuição das idades dos 1.000 repositórios, com indicação da mediana e da média.

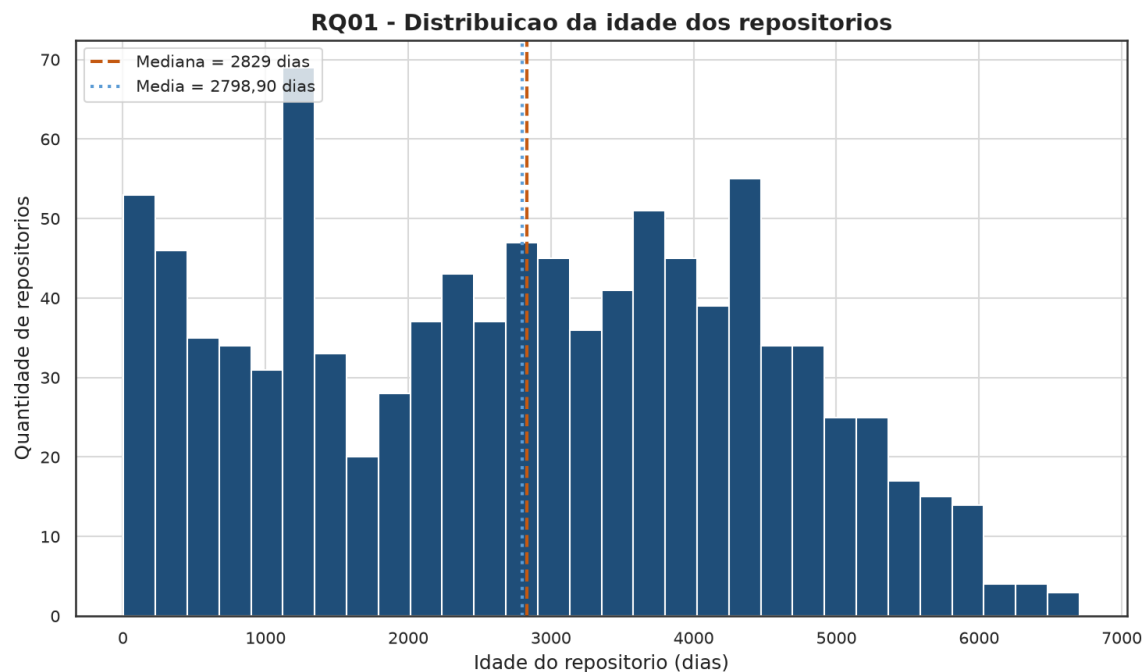


Figura 1 — Distribuição da idade dos repositórios analisados. Fonte: Elaboração própria.

A idade mediana foi de 2.829 dias, aproximadamente 7,75 anos, enquanto a média foi de 2.798,90 dias, aproximadamente 7,67 anos. Os valores variaram entre 5 e 6.703 dias. A proximidade entre média e mediana e a concentração visual em projetos com vários anos de existência sustentam a conclusão de que os repositórios populares analisados tendem a ser maduros. Não foram identificados outliers de idade pelo método IQR.

4.2.2 RQ02 — Pull Requests aceitas

RQ02: Sistemas populares recebem muita contribuição externa?

A contribuição foi representada pelo total de Pull Requests no estado MERGED. A Figura 2 utiliza um boxplot com escala symlog, que mantém os valores iguais a zero e permite visualizar projetos com volumes extremamente elevados.

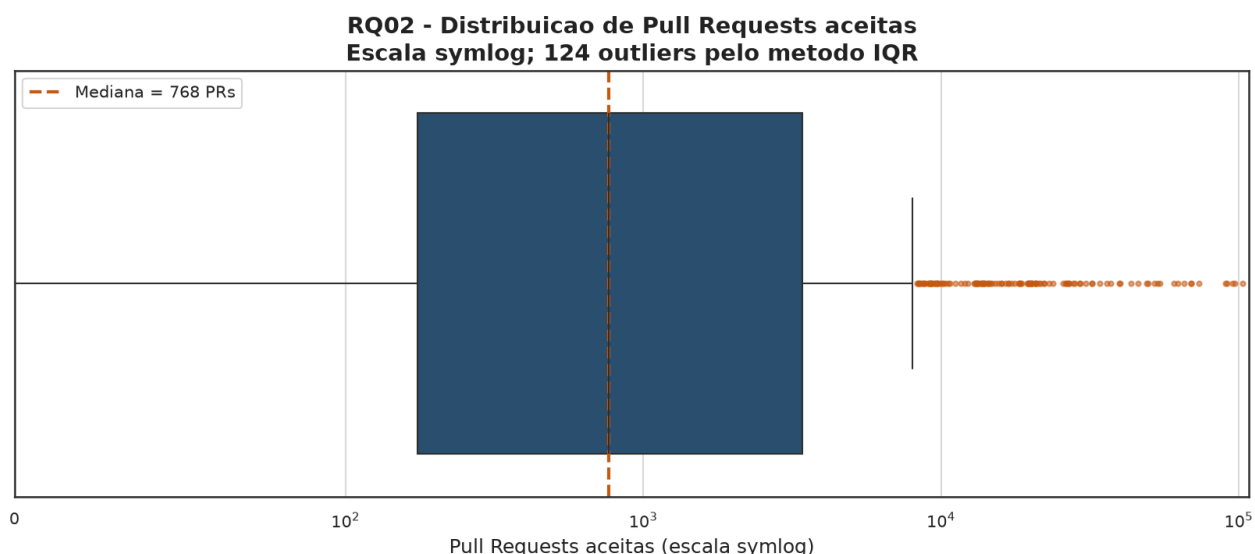


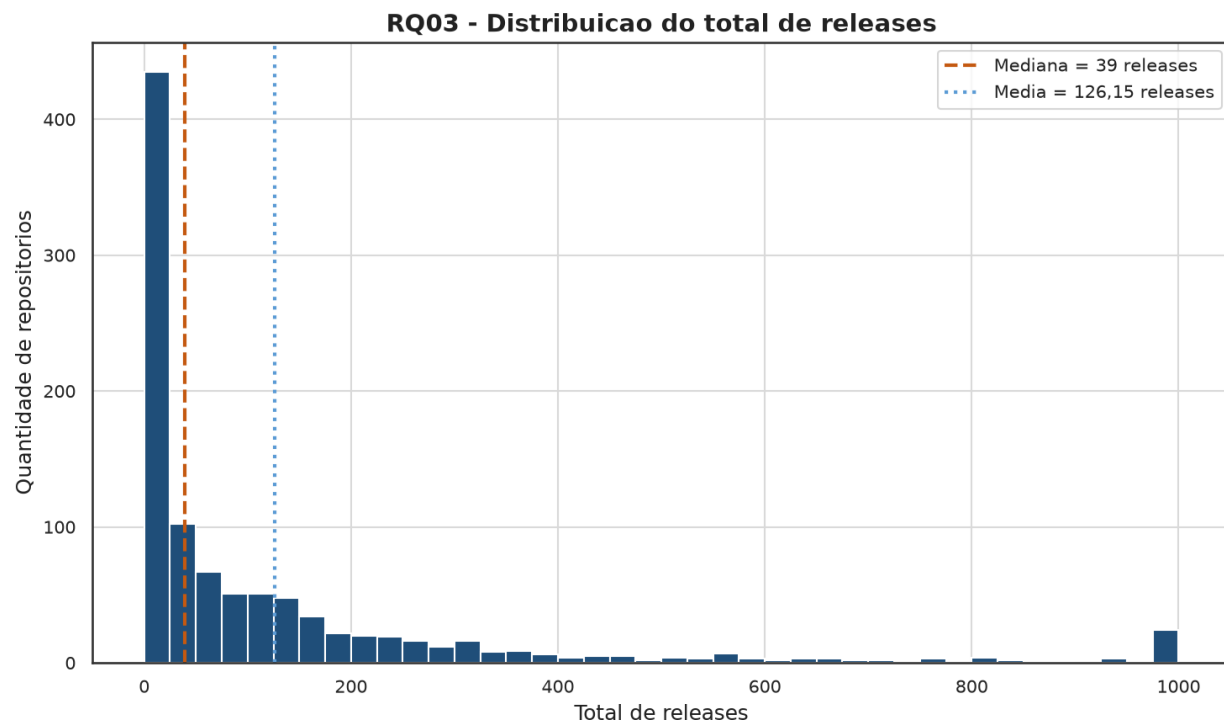
Figura 2 — Distribuição das Pull Requests aceitas nos repositórios. Fonte: Elaboração própria.

A mediana foi de 768 Pull Requests aceitas, enquanto a média foi de 4.233,82. Foram identificados 124 possíveis outliers pelo método IQR. Além disso, 819 repositórios possuíam pelo menos 100 Pull Requests aceitas, 452 possuíam pelo menos 1.000 e 20 apresentaram valor igual a zero. A diferença entre média e mediana confirma uma distribuição fortemente assimétrica, na qual poucos projetos concentram volumes muito elevados.

4.2.3 RQ03 — Releases

RQ03: Sistemas populares lançam releases com frequência?

A métrica utilizada foi o total acumulado de releases registrado pelo GitHub. A Figura 3 apresenta sua distribuição nos 1.000 repositórios, com indicação da mediana e da média.



RQ04 - Comparacao updatedAt vs pushedAt
Limitacao: updatedAt muda com qualquer atividade e quase nao discrimina

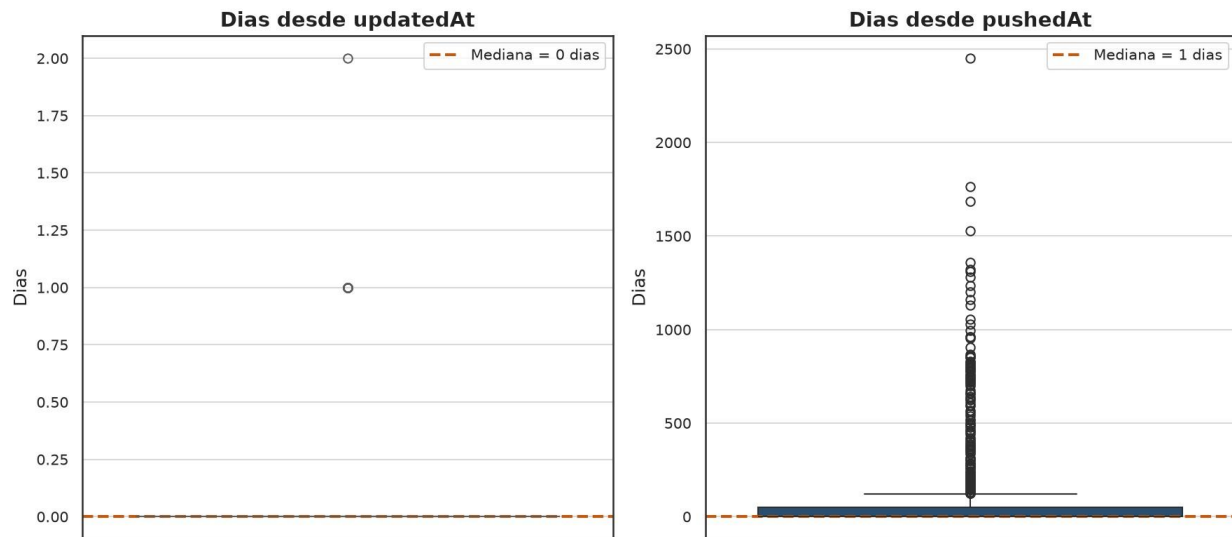


Figura 4 — Comparação dos dias desde updatedAt e pushedAt. Fonte: Elaboração própria.

O campo updatedAt apresentou mediana de zero dias e baixa capacidade de diferenciação entre os projetos. Em contraste, pushedAt apresentou mediana de um dia e maior dispersão. Foram observados 620 repositórios com push nos sete dias anteriores à coleta, 725 nos últimos 30 dias e 790 nos últimos 90 dias. Esses resultados indicam que a maioria dos repositórios apresentava atividade recente, mas também confirmam que pushedAt é uma medida mais adequada para representar alterações no conteúdo

4.2.5 RQ05 — Linguagens

RQ05: Sistemas populares são escritos em linguagens populares?

A linguagem primária foi obtida pelo campo primaryLanguage do GitHub. As linguagens foram comparadas com o conjunto das dez linguagens mais populares adotado a partir do GitHub Octoverse 2024.

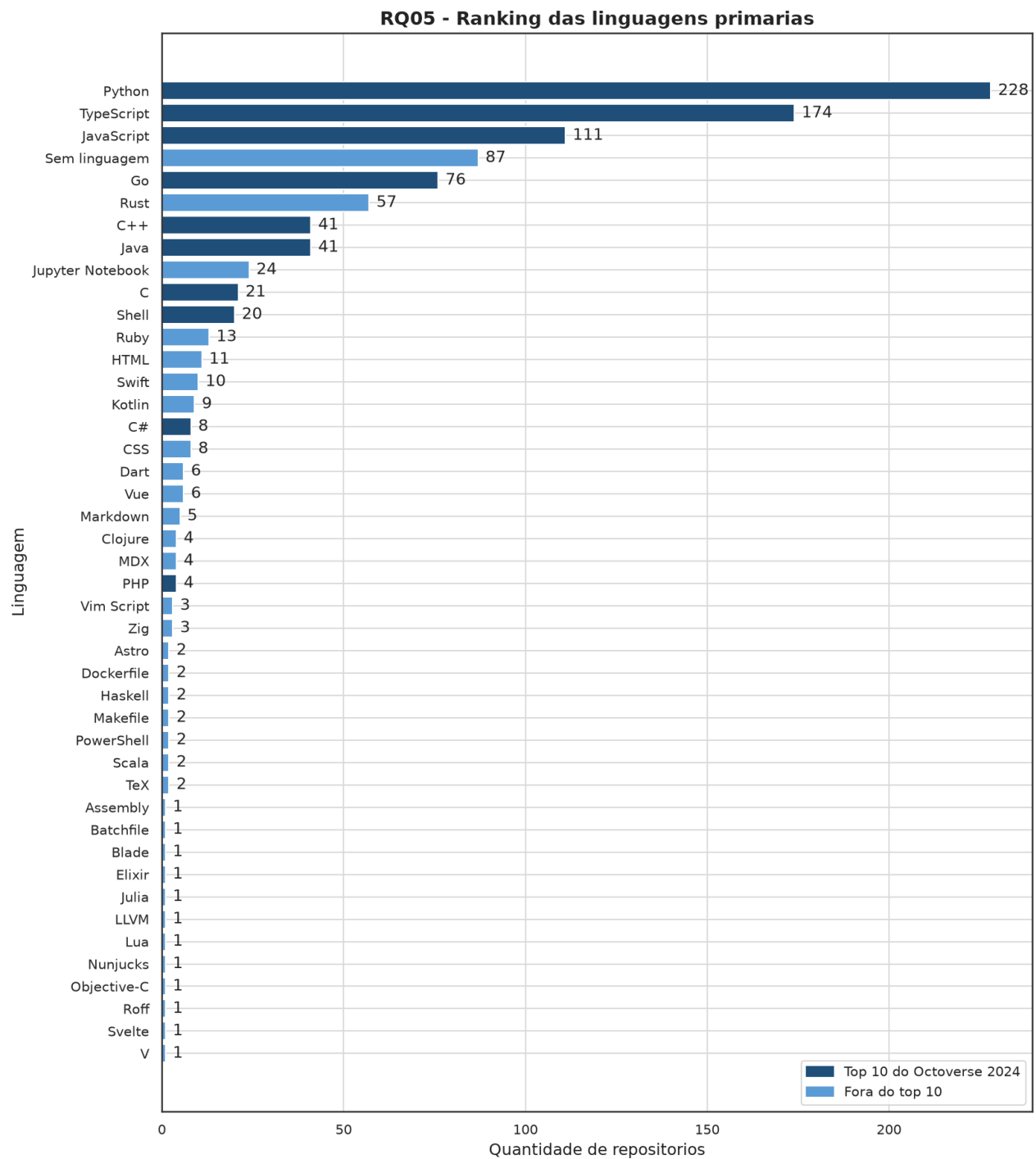


Figura 5 — Distribuição das linguagens primárias dos repositórios. Fonte: Elaboração própria.

Python foi a linguagem mais frequente, com 228 repositórios. Ao todo, 724 dos 1.000 projetos utilizavam uma linguagem pertencente ao top 10 adotado. Entre os 913 repositórios com linguagem identificada, isso corresponde a aproximadamente 79,3%. Outros 87 projetos não possuíam linguagem primária identificada e foram mantidos na análise por representarem casos reais, como listas, materiais educacionais e repositórios de documentação.

4.2.6 RQ06 — Issues Fechadas

RQ06: Sistemas populares possuem alto percentual de issues fechadas?

O percentual foi calculado pela divisão entre o total de issues fechadas e o total de issues do repositório. Projetos sem issues não participaram do cálculo da proporção.

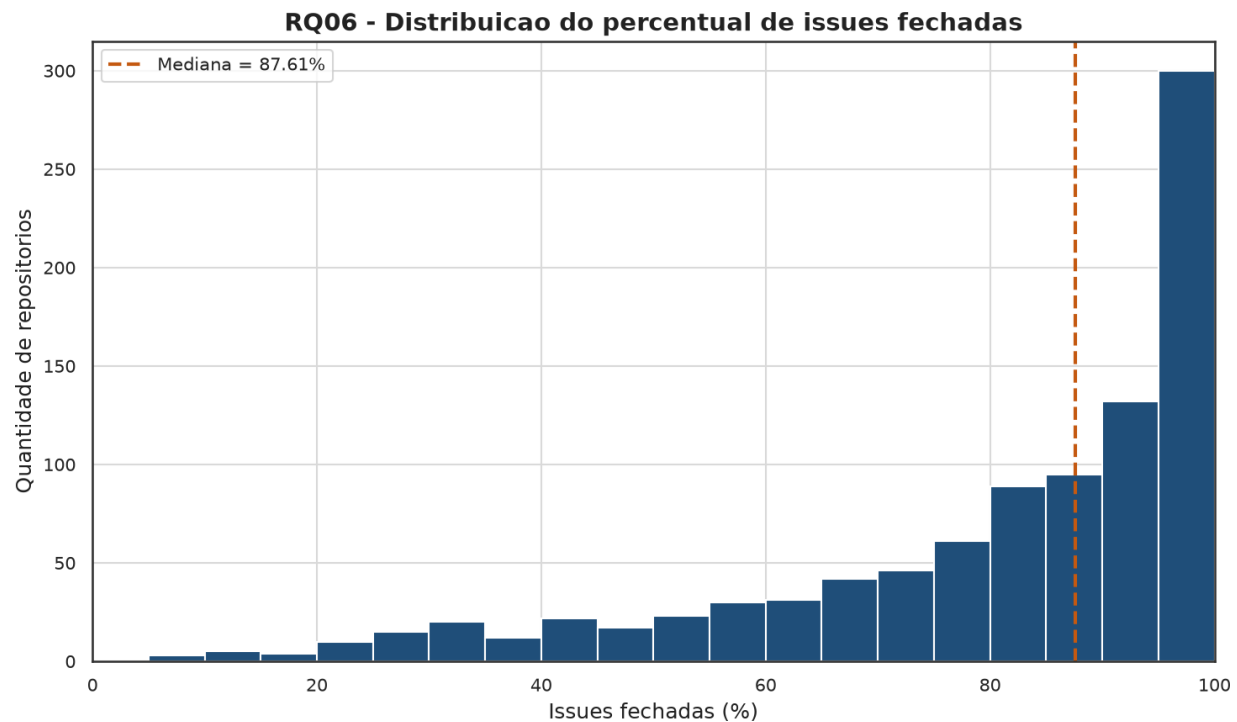


Figura 6 — Distribuição do percentual de issues fechadas. Fonte: Elaboração própria.

Entre os 957 repositórios com issues, a mediana do percentual de encerramento foi de 87,61%. O primeiro quartil foi de 70,42% e o terceiro quartil foi de 96,84%. Ao todo, 677 repositórios atingiram pelo menos 75% de issues fechadas e 432 atingiram pelo menos 90%. Os outros 43 projetos não possuíam issues e, portanto, não participaram dessa distribuição. Os resultados sustentam a hipótese de que projetos populares encerram uma proporção elevada das issues registradas.

4.2.7 RQ07 — Métricas por linguagem

RQ07: Repositórios escritos em linguagens populares apresentam mais contribuições, releases e atualizações?

Os repositórios foram agrupados por linguagem primária. Para cada grupo, foram calculadas as medianas de Pull Requests aceitas, releases e dias desde a última atualização. O tamanho de cada ponto representa a quantidade de projetos pertencentes ao grupo.

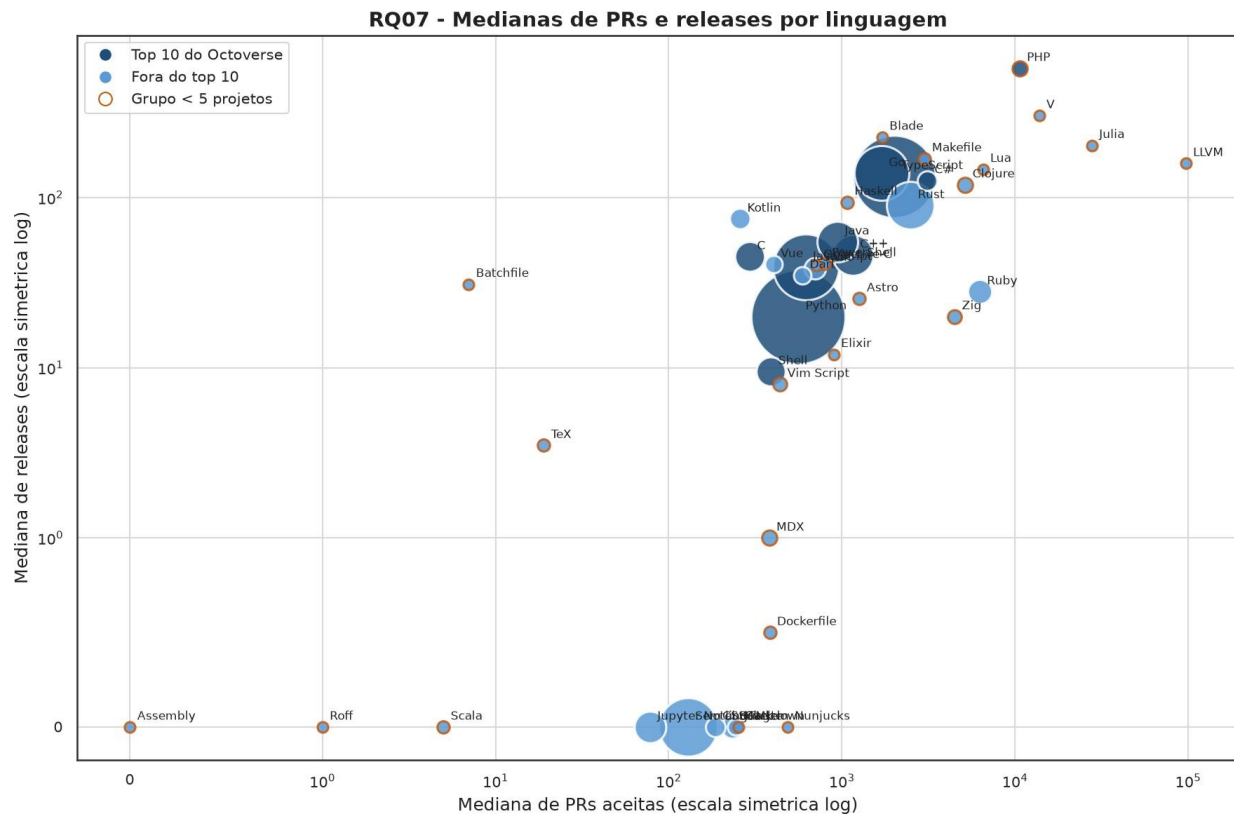


Figura 7 — Relação entre as medianas de Pull Requests e releases por linguagem. Fonte: Elaboração própria.

O maior grupo foi Python, com 228 repositórios. Algumas linguagens, como TypeScript, Go e Rust, apresentaram medianas elevadas de Pull Requests e releases. Entretanto, também foram observados valores elevados em linguagens representadas por poucos projetos. Foram identificados 24 grupos com menos de cinco repositórios, destacados por borda diferenciada no gráfico. Dessa forma, os resultados não permitem afirmar que a popularidade da linguagem, isoladamente, determine maior contribuição ou frequência de releases.

4.2.8 Validação automática das figuras

Todas as figuras foram submetidas a uma validação automática após sua geração. O processo verificou a existência e o tamanho não vazio dos sete arquivos, a assinatura do formato PNG, a resolução mínima, a presença dos títulos, a identificação dos eixos e das unidades e a correspondência entre as métricas apresentadas e os resultados calculados sobre os 1.000 repositórios. As sete figuras foram aprovadas, e o resumo da validação foi armazenado no arquivo `reports/validacao-figuras.md`.

4.3 Discussão

Os resultados foram analisados de forma descritiva, utilizando principalmente medianas, quartis, médias, contagens e identificação de possíveis outliers. Não foram aplicados testes estatísticos

inferenciais nesta etapa, pois o objetivo do Lab01 é caracterizar os 1.000 repositórios mais populares, e não comparar amostras experimentais ou estabelecer relações causais.

Na RQ01, a hipótese de que repositórios populares tendem a ser maduros foi confirmada. A idade mediana foi de 2.829 dias, aproximadamente 7,75 anos, e a média foi de 2.798,90 dias, aproximadamente 7,67 anos. A proximidade entre média e mediana indica que a conclusão não é resultado apenas de poucos projetos muito antigos. Embora existam repositórios recentes entre os mais estrelados, os resultados mostram que a maior parte dos projetos populares acumulou vários anos de existência.

Na RQ02, a hipótese de que repositórios populares recebem muitas contribuições externas foi confirmada, com ressalvas. A mediana foi de 768 Pull Requests aceitas, 819 repositórios apresentaram pelo menos 100 e 452 apresentaram pelo menos 1.000. A média de 4.233,82 foi muito superior à mediana, e foram identificados 124 possíveis outliers, confirmando a hipótese de uma distribuição assimétrica, na qual poucos projetos concentram volumes extremamente elevados. Entretanto, a métrica representa Pull Requests aceitas e não permite distinguir automaticamente contribuições externas de Pull Requests criadas por integrantes da própria organização mantenedora.

Na RQ03, a hipótese de que sistemas populares lançam releases com frequência foi parcialmente confirmada. A mediana foi de 39 releases e 644 repositórios apresentaram pelo menos 10, indicando que muitos projetos utilizam esse mecanismo regularmente. Por outro lado, 286 repositórios não possuíam nenhuma release registrada, e o primeiro quartil também foi igual a zero. Além disso, o total acumulado de releases não mede diretamente frequência, pois não considera a idade do projeto ou o intervalo entre versões. Dessa forma, popularidade não implica necessariamente adoção do mecanismo de releases do GitHub.

Na RQ04, a hipótese de que projetos populares são atualizados frequentemente foi confirmada quando utilizado o campo `pushedAt`, mas o resultado baseado somente em `updatedAt` foi inconclusivo. O campo `updatedAt` apresentou mediana de zero dias, com 992 repositórios atualizados no dia da coleta, demonstrando baixa capacidade de diferenciação. Já `pushedAt` apresentou mediana de 1 dia, com 620 projetos atualizados nos últimos 7 dias, 725 nos últimos 30 dias e 790 nos últimos 90 dias. Esses resultados indicam atividade recente na maior parte da amostra, embora existam projetos com longos períodos sem push, incluindo 194 possíveis outliers.

Na RQ05, a hipótese de que repositórios populares utilizam linguagens também populares foi confirmada. Entre os 1.000 projetos, 724 utilizam uma das dez linguagens consideradas mais populares pelo GitHub Octoverse 2024. Quando são considerados somente os 913 repositórios com linguagem identificada, esse resultado corresponde a aproximadamente 79,3%. Python, TypeScript e JavaScript foram as três linguagens mais frequentes, somando 513 repositórios. Os 87 casos sem linguagem primária foram mantidos, pois representam projetos documentais, listas e outros conteúdos que também fazem parte dos repositórios mais populares.

Na RQ06, a hipótese de que sistemas populares apresentam alto percentual de issues fechadas foi confirmada. Entre os 957 repositórios para os quais a razão pôde ser calculada, a mediana foi de 87,61%, o primeiro quartil foi de 70,42% e o terceiro quartil foi de 96,84%. Esses valores indicam que a maioria dos projetos fecha uma parcela elevada das issues registradas. Contudo, a proporção de issues

fechadas não informa se uma solicitação foi resolvida satisfatoriamente, recusada, considerada duplicada ou encerrada automaticamente. Portanto, a métrica representa capacidade de encerramento, mas não necessariamente qualidade da resolução.

Na RQ07, a hipótese de que linguagens populares apresentam maior contribuição externa e mais releases foi parcialmente confirmada. TypeScript, Go e Rust apresentaram medianas elevadas de Pull Requests aceitas e releases, enquanto Python e JavaScript, apesar de muito frequentes, apresentaram valores menores em algumas comparações. Também foram observadas linguagens com medianas elevadas, mas representadas por poucos projetos, como PHP, Clojure, Julia e LLVM. Por esse motivo, não é possível concluir que a popularidade da linguagem, isoladamente, determina maior contribuição ou frequência de releases. A comparação de atualização por `updatedAt` foi inconclusiva porque todas as linguagens apresentaram mediana de zero dias. A análise complementar com `pushedAt` oferece maior capacidade de diferenciação, mas deve considerar a quantidade desigual de projetos em cada grupo.

Ameaças à validade

Uma ameaça à validade está relacionada à definição de popularidade. O número de estrelas foi utilizado como critério de seleção, mas estrelas representam interesse ou visibilidade e não necessariamente qualidade, uso em produção ou atividade de manutenção. A ordenação também é dinâmica: novas estrelas e novos projetos podem alterar a composição dos 1.000 primeiros repositórios em coletas futuras.

A coleta foi executada por paginação durante vários minutos. Como o GitHub permanece ativo durante esse período, contagens de estrelas, issues, Pull Requests e releases podem sofrer pequenas alterações entre a primeira e a última página. Assim, a base representa um retrato aproximado do período da coleta, não um estado simultâneo e imutável da plataforma.

Algumas métricas possuem limitações operacionais. O total de releases não considera a idade do projeto e, portanto, não mede diretamente a frequência entre versões. Pull Requests aceitas não distinguem automaticamente participantes externos de integrantes das organizações mantenedoras. O campo `primaryLanguage` é calculado pelo GitHub e pode não representar projetos multilíngues. A proporção de issues fechadas não informa a razão ou a qualidade do encerramento. O campo `updatedAt` pode ser alterado por atividades que não envolvem código, o que motivou a inclusão de `pushedAt`.

A utilização do GitHub Octoverse 2024 em uma coleta realizada em 2026 também representa uma limitação temporal. A fonte foi mantida porque havia sido definida no início do laboratório, garantindo consistência metodológica, mas a posição relativa das linguagens pode ter mudado posteriormente.

Na RQ07, os grupos possuem tamanhos muito diferentes. Python possui 228 repositórios, enquanto diversas linguagens possuem menos de cinco. Valores elevados em grupos pequenos podem refletir um único projeto excepcional e não uma característica geral da linguagem. Por isso, esses grupos foram sinalizados e interpretados com cautela. As relações observadas são descritivas e não devem ser interpretadas como evidência de causalidade.

Contribuições das inovações

As inovações propostas pelo grupo aprofundaram os resultados previstos no enunciado. A inclusão de `pushedAt` mostrou que o resultado de `updatedAt` era pouco discriminante e tornou a RQ04 mais confiável. A validação automática confirmou que os 1.000 registros eram únicos e que os campos obrigatórios estavam consistentes, além de documentar linguagens ausentes e repositórios sem issues. A identificação de outliers pelo método IQR explicou diferenças elevadas entre média e mediana, especialmente em Pull Requests e releases, evitando conclusões baseadas apenas na média. Por fim, a marcação de grupos pequenos nas análises por linguagem reduziu o risco de generalizar resultados produzidos por poucos projetos. Assim, as contribuições adicionais não contradisseram os resultados obrigatórios, mas aumentaram sua precisão, transparência e capacidade de interpretação.

5. Conclusão

A análise dos 1.000 repositórios mais populares do GitHub mostrou que esses projetos tendem a ser maduros, recebem volumes relevantes de Pull Requests e geralmente apresentam alta proporção de issues fechadas. A publicação de releases, entretanto, não é uma prática uniforme: alguns projetos acumulam muitas versões, enquanto uma parcela significativa não utiliza o mecanismo de releases do GitHub. Também foi observada forte concentração em linguagens amplamente utilizadas, principalmente Python, TypeScript e JavaScript, confirmando que popularidade de linguagem e visibilidade dos projetos possuem alguma associação.

A maioria dos repositórios apresentou atividade recente, mas a comparação entre `updatedAt` e `pushedAt` demonstrou que a escolha da métrica altera a interpretação da frequência de atualização. O primeiro campo reflete diferentes tipos de atividade no GitHub e apresentou pouca variação, enquanto o segundo representou melhor as alterações no conteúdo. Nas análises agrupadas, algumas linguagens apresentaram maior número de contribuições e releases, mas não foi identificado um padrão suficiente para afirmar que linguagens mais populares sempre produzem projetos mais ativos. As diferenças de tamanho entre os grupos e a presença de projetos extremos exigem cautela nessas comparações.

As principais limitações estão relacionadas ao uso de estrelas como representação de popularidade, à natureza dinâmica dos dados do GitHub e às definições operacionais das métricas. Pull Requests aceitas não distinguem perfeitamente contribuições externas de atividades dos mantenedores; o total de releases não mede diretamente o intervalo entre versões; a linguagem primária não representa toda a composição de projetos multilíngues; e o encerramento de uma issue não garante que ela tenha sido resolvida satisfatoriamente. Além disso, o ranking do GitHub Octoverse 2024 foi aplicado a uma coleta realizada em 2026 para preservar a consistência metodológica definida no início do laboratório. Embora a amostra de 1.000 repositórios seja ampla, ela representa somente os projetos com maior número de estrelas e não pode ser generalizada para todos os projetos de software.

As inovações propostas pelo grupo aumentaram a confiabilidade e a capacidade de interpretação do estudo. A inclusão de `pushedAt` aprofundou a análise de atualização, a validação automática reduziu o risco de analisar dados inconsistentes, a identificação de outliers explicou distribuições assimétricas e

a marcação de grupos pequenos evitou generalizações indevidas nas comparações por linguagem. Com mais tempo e recursos, o grupo ampliaria a coleta para diferentes períodos, permitindo analisar a evolução das métricas, normalizaria releases e contribuições pela idade de cada projeto, identificaria a origem interna ou externa dos autores das Pull Requests e aplicaria técnicas estatísticas para avaliar as diferenças entre linguagens. Entre as contribuições adicionais, o acompanhamento temporal de pushedAt e a análise automatizada de qualidade dos dados são as inovações com maior potencial para expansão em trabalhos futuros.

6. Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- GITHUB. Octoverse 2024: AI leads Python to top language as the number of global developers surges. GitHub Blog, 2024. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-2024/>. Acesso em: 19 ago. 2026.
- GITHUB. Planning and tracking with Projects. GitHub Docs. Disponível em: <https://docs.github.com/en/issues/planning-and-tracking-with-projects>. Acesso em: 19 ago. 2026.
- GITHUB. Using pagination in the GraphQL API. GitHub Docs. Disponível em: <https://docs.github.com/en/graphql/guides/using-pagination-in-the-graphql-api>. Acesso em: 19 ago. 2026.
- TUKEY, John W. Exploratory data analysis. Reading: Addison-Wesley Publishing Company, 1977.